BÖLÜM 4 / MODULE 4

Veritabanı Mimarisi, Bölümleme ve Replikasyon

Database Architecture, Sharding & Replication Topologies

SQL vs NoSQL seçim matrisi, NoSQL türleri, Single/Multi-Leader replikasyon, Sharding stratejileri, Consistent Hashing ve B-Tree vs LSM-Tree indeksleme motorları.

SQL vs NoSQL trade-offs, NoSQL families, Quorum replication, Sharding topologies, Consistent Hashing rings, and B-Tree vs LSM-Tree storage engines.

19

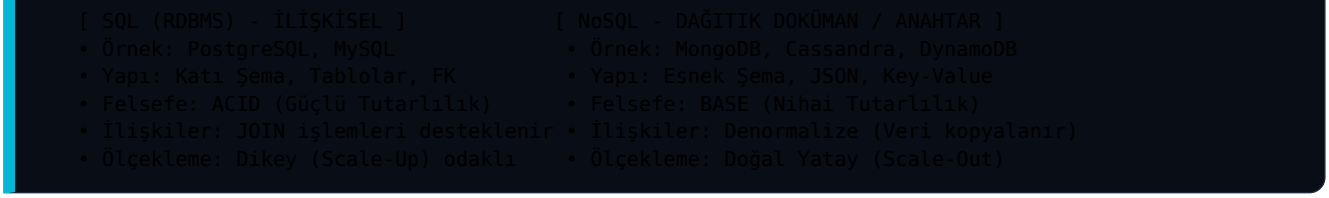
BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

İlişkisel (SQL) vs NoSQL: ACID vs BASE ve Doğru Veri Modeli Seçimi

SQL vs NoSQL: ACID vs BASE & Picking the Right Data Store

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

SQL veritabanı, her satırı ve sütunu cetvelle çizilmiş, katı kuralları olan bir devlet tapu sicil dairesi gibidir: Her şey kanunlara (ACID) uyar; iki tapu birbiriyle çakışamaz (JOIN), ama defter büyüdükçe binaya yeni kat çıkmak (Dikey ölçekleme) zorunda kalırsınız. NoSQL ise devasa bir kargo aktarma merkezindeki pratik plastik koliler gibidir (BASE): Her kolinin içine istediğiniz şekilde eşya koyarsınız (esnek JSON şeması); 1 milyon koli gelirse yan tarafa 100 tane yeni depo inşa edip (Yatay ölçekleme) kolileri depolara fırlatırsınız!

🇬🇧 ENGLISH (ELI5)

A relational SQL database is a notarized government land registry: strictly structured tables, foreign keys, and absolute legal validity (ACID guarantees). Everything cross-references seamlessly via JOINs, but expanding means building a taller skyscraper (vertical scaling). NoSQL is an automated Amazon fulfillment sorting warehouse: shipping containers packed with self-contained JSON records (BASE philosophy). When traffic explodes, you don't build taller; you roll out 100 new commodity warehouse bays (horizontal scale-out)!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) SQL (ACID): Atomicity, Consistency, Isolation, Durability. Finansal işlemler, e-ticaret ödemeleri ve karmaşık ilişkisel sorgular (JOIN) için vazgeçilmezdir. Zorluğu: Milyarlarca satıra ulaştığında sharding ve cross-node join yapmak aşırı zordur. 2) NoSQL (BASE): Basically Available, Soft state, Eventual consistency. Şemasızdır (Schema-on-read); yatayda onlarca sunucuya yayılmak üzere tasarlanmıştır. Veriler sorgu desenine göre denormalize edilir (JOIN yapılmaz, veriler iç içe gömülür). Seçim Kriteri: Veriniz yüksek ilişkisel mi, yoksa saniyede yüzbinlerce okuma/yazma gerektiren analitik/akış verisi mi?

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) SQL (ACID): Strict schemas with normalized relations, foreign key constraints, and transactional isolation. Essential for financial ledgers, transactional checkout baskets, and complex relational graphs. Scaling ceiling: Cross-shard joins are computationally prohibitive and multi-master clustering introduces heavy consensus locking. 2) NoSQL (BASE): Basically Available, Soft State, Eventual Consistency. Schemaless flexibility (schema-on-read) built ground-up for horizontal sharding. Documents denormalize data to prioritize O(1) primary key queries over cross-table joins.

⚡ Modern Java 25 Uygulaması (Loom & Records) / ⚡ Java 25 Implementation

```
// Java 25: SQL (Normalized) vs NoSQL (Denormalized) Veri Modeli
public class DataModelComparison {

    // 1. SQL Tarzı: Normalize Edilmiş İlişkisel Yapı (Ayrı Tablolar + ID Referansı)
    public record SqlOrder(long orderId, long customerId, java.math.BigDecimal totalAmount) {}
    public record SqlOrderItem(long itemId, long orderId, String productName, int quantity) {}

    // 2. NoSQL Tarzı: Denormalize Edilmiş Doküman (Tek Parça, Sıfır JOIN!)
    public record NoSqlOrderDocument(
        String orderId,
        String customerName,
        String customerEmail,
        java.util.List<OrderItemDoc> items,
        java.math.BigDecimal totalAmount
    ) {
        public record OrderItemDoc(String product, int qty, java.math.BigDecimal price) {}
    }
}

void main() {
    var item = new DataModelComparison.NoSqlOrderDocument.OrderItemDoc("MacBook Pro M4", 1, new
java.math.BigDecimal("2500"));
    var doc = new DataModelComparison.NoSqlOrderDocument("ord_101", "Ali Bozkurt", "ali@test.com",
java.util.List.of(item), new java.math.BigDecimal("2500"));
    IO.println("NoSQL Tekil Doküman Oluşturuldu: ID=" + doc.orderId() + ", Ürün=" +
doc.items().getFirst().product());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı / ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatçının 'Hangi veritabanını seçersin?' sorusuna hemen 'MongoDB!' veya 'PostgreSQL!' diye atlamayın. Önce veri erişim modelini (Access Pattern) analiz edin: 'Veri ilişkisel mi? Güçlü ACID gerekiyor mu? Yoksa yüksek hacimli zaman serisi veya anahtar-değer erişimi mi var?' diyerek gerekçenizi temellendirin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never blurt out 'I'll use MongoDB!' before analyzing the query access patterns. State the decision matrix: 'Because our workload requires complex relational multi-table transactions, I recommend PostgreSQL' or 'Because our writes hit 500K QPS with simple key-value reads, Cassandra is optimal.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar / 🎯 Key Takeaways for the Interview

- Finansal tutarlılık ve zengin JOIN sorguları için PostgreSQL/MySQL tercih edilir.
- Yüksek hacimli, şeması değişken ve yatay ölçeklenebilir veriler için NoSQL seçilir.
- Modern mimarilerde Hibrit (Polyglot Persistence) yaklaşım kullanılır (ör. Siparişler SQL'de, Akış Cassandra'da).

- RDBMS (PostgreSQL/MySQL) remains optimal for transactional ACID and relational queries.
- NoSQL fits high-throughput, denormalized, horizontally scalable workloads.
- Enterprise architectures adopt Polyglot Persistence: SQL for orders, NoSQL for feeds.

20

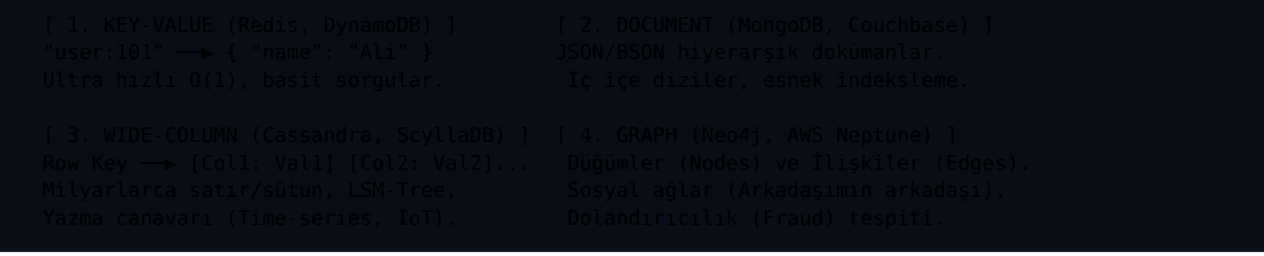
BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

NoSQL Aileleri: Document, Key-Value, Wide-Column (Cassandra) ve Graph

The 4 NoSQL Families: Document, Key-Value, Wide-Column & Graph

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

4 farklı alet çantasını düşünün: 1) Key-Value (Redis): Otel vestiyeri gibidir; fiş numarasını verirsiniz, ceketi saniyesinde alırsınız (hızlı ama ceketin cebinde ne olduğunu arayamazsınız). 2) Document (MongoDB): Lüks bir dosya klasörüdür; içinde müşterinin tüm faturaları, fotoğrafları tek bir dosyadadır. 3) Wide-Column (Cassandra): Sonsuz genişlikte devasa bir muhasebe tablosudur; saniyede 1 milyon satır faturayı tıkır tıkır kaydeder (yazma şampiyonu). 4) Graph (Neo4j): Dedektiflerin panosundaki iplerle birbirine bağlanmış fotoğraflardır; 'Ali'nin arkadaşının takip ettiği sayfaları beğenen kim var?' sorusunu şıp diye çözer!

🇬🇧 ENGLISH (ELI5)

Four specialized power tools: 1) Key-Value (Redis, DynamoDB): A coat check room; you hand over ticket 42, you get coat 42 in 1 millisecond (super fast, but you cannot search coats by collar size). 2) Document (MongoDB): A manila folder storing self-contained rich JSON documents with nested arrays. 3) Wide-Column (Cassandra): An infinite spreadsheet engineered with LSM-Trees that swallows 1,000,000 writes/sec without sweating (IoT, telemetry). 4) Graph (Neo4j): A detective's corkboard with red yarn connecting suspects (Nodes) and crimes (Edges); answering 'Who is a friend-of-a-friend that likes this page?' in milliseconds!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

NoSQL 4 ana kategoriye ayrılır: 1) Key-Value: En basit, en hızlı veri modeli (Redis, Memcached, DynamoDB). Anahtar üzerinden O(1) okuma/yazma. 2) Document Store: JSON/BSON dokümanları (MongoDB, Couchbase). İkincil indeksler (Secondary Indexes) desteklenir; e-ticaret ürün katalogları ve içerik yönetim sistemleri için uygundur. 3) Wide-Column (Columnar Store): Google Bigtable ve Apache Cassandra mimarisi. Veriler sütun aileleri (Column Families) ve Row Key bazında LSM-Tree ile saklanır. Yüksek yazma verimi (Write Throughput) ve zaman serisi (Metrics, Chat geçmişi) için liderdir. 4) Graph Database: Düğümler (Nodes), kenarlar (Edges) ve özellikler (Properties) (Neo4j, Amazon Neptune). Çok katmanlı ilişkisel gezinme (Graph Traversal) sorgularında SQL JOIN'lerinin aksine performans kaybı yaşamaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Key-Value (Redis, DynamoDB): Pure O(1) hash maps keyed by unique strings. Minimalist, sub-millisecond, highly partitionable. 2) Document (MongoDB, Couchbase): Semi-structured JSON trees permitting rich secondary indexes and aggregation pipelines, ideal for polymorphic product catalogs. 3) Wide-Column (Cassandra, ScyllaDB): Bigtable-derived sparse multidimensional sorted maps. Optimized with append-only LSM-Trees for extreme write throughput and linear horizontal scaling across IoT feeds, audit logs, and message timelines. 4) Graph (Neo4j, Neptune): Pointer-chasing graph traversal engine executing deep relationship pathfinding (fraud rings, social graphs, recommendation networks) without explosive multi-way SQL JOIN overhead.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Graph Veritabanı Mantığı (Düğümler ve İlişkiler Simülasyonu)
import java.util.*;

public class GraphDatabaseSim {
    public record Node(String id, String name) {}
    public record Edge(Node from, Node to, String relation) {}

    private final Map<Node, List<Edge>> adjacencyList = new HashMap<>();

    public void addFriendship(Node a, Node b) {
        adjacencyList.computeIfAbsent(a, k -> new ArrayList<>()).add(new Edge(a, b, "FRIENDS_WITH"));
        adjacencyList.computeIfAbsent(b, k -> new ArrayList<>()).add(new Edge(b, a, "FRIENDS_WITH"));
    }

    public List<Node> getFriendsOfFriends(Node person) {
        Set<Node> fofs = new HashSet<>();
        for (Edge friendEdge : adjacencyList.getOrDefault(person, List.of())) {
            Node friend = friendEdge.to();
            for (Edge fofEdge : adjacencyList.getOrDefault(friend, List.of())) {
                if (!fofEdge.to().equals(person)) {
                    fofs.add(fofEdge.to());
                }
            }
        }
        return new ArrayList<>(fofs);
    }
}

void main() {
    var graph = new GraphDatabaseSim();
    var ali = new GraphDatabaseSim.Node("1", "Ali");
    var bob = new GraphDatabaseSim.Node("2", "Bob");
    var can = new GraphDatabaseSim.Node("3", "Can");

    graph.addFriendship(ali, bob);
    graph.addFriendship(bob, can); // Ali -> Bob -> Can

    IO.println("Ali'nin Arkadaşının Arkadaşları: " +
graph.getFriendsOfFriends(ali).getFirst().name());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Cassandra gibi Wide-Column sistemlerde 'İstediğim sütuna göre sonradan SELECT WHERE çekerim' diyemezsiniz! Cassandra'da sorgular Partition Key ve Clustering Key'e göre KATI bir şekilde önceden modellenmelidir; sorgu desenini bilmeden tablo tasarlanamaz.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

In Cassandra, you cannot run arbitrary ad-hoc queries with WHERE clauses on non-key columns without table scans. In Wide-Column stores, table schemas are rigidly designed around explicit query access patterns.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Key-Value basit nesneler ve oturumlar için en hızlı NoSQL türüdür.

• Cassandra LSM-Tree yapısı sayesinde saniyede milyonlarca yazmayı kayıpsız yutar.

• Sosyal ağ ve dolandırıcılık tespiti gibi ilişkisel ağlarda Graph DB rakipsizdir.

🎯 Key Takeaways for the Interview

• Key-Value stores offer unmatched speed for simple session and payload caches.

• Cassandra's append-only LSM-Tree architecture excels at high-velocity writes.

• Graph databases conquer multi-hop relationship pathfinding with zero JOIN degradation.

21

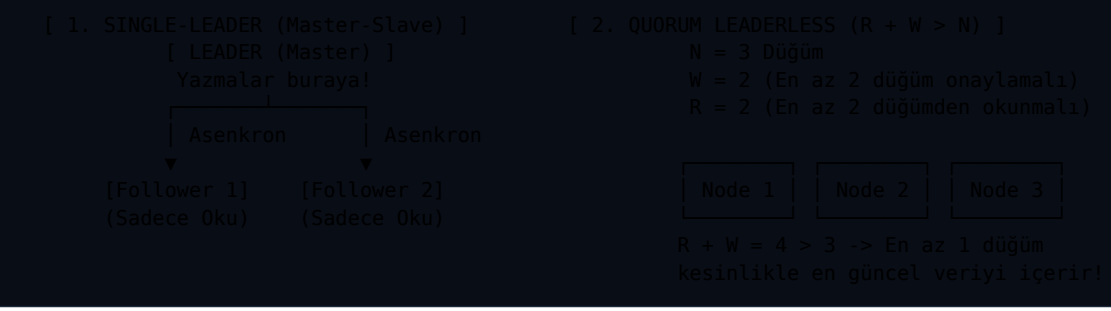
BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

Veritabanı Replikasyonu: Single-Leader, Multi-Leader ve Quorum (R + W > N)

Replication Topologies: Single-Leader, Multi-Leader & Quorum (R+W > N)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Single-Leader replikasyon, sınıfta tek bir öğretmenin (Leader/Master) konuşması ve 30 öğrencinin (Followers/Replicas) öğretmeni defterine kopyalaması gibidir. Herkes soru sorup okuyabilir (Okuma ölçeklenir), ama tahtaya sadece öğretmen yazar. Öğretmen bayılırsa öğrencilerden biri yeni öğretmen seçilir (Failover). Quorum (Lidersiz) ise 3 kişilik bir jüri heyetidir: Bir kararın geçerli olması için en az 2 üyenin 'Evet' demesi (Write Quorum: W=2) ve okurken en az 2 üyeye danışılması (Read Quorum: R=2) şarttır. İki çoğunluk kümesi mutlaka en az 1 kişide kesişeceği için doğru bilgi asla kaybolmaz!

🇬🇧 ENGLISH (ELI5)

Single-Leader replication is a lecture hall where only the professor (Leader/Master) speaks and 30 students (Followers/Replicas) transcribe notes into their notebooks. Anyone can read, but only the professor writes. If the professor faints, a student is elected the new professor (Failover). Leaderless Quorum (Dynamo/Cassandra) is a 3-judge jury: to confirm a fact, at least 2 judges must sign off (W=2), and to read, at least 2 judges must be polled (R=2). Because two majorities mathematically overlap, you are guaranteed to read the latest truth!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) Single-Leader (Primary-Secondary): Tüm yazmalar Leader düğüme gider; Leader değişiklikleri replikasyon logu (WAL) ile follower'lara asenkron veya yarı-senkron aktarır. Okuma trafiği follower'lara dağıtılarak ölçeklenir; ancak 'Replication Lag' (gecikme) nedeniyle 'Kendi Yazdığını Okuyamama' (Read-Your-Own-Writes) sorunu oluşabilir. 2) Multi-Leader: Birden fazla veri merkezinde yazma kabul edilir; ancak eşzamanlı çakışmalar (Write Conflicts) çözülmelidir (Last-Write-Wins veya CRDT). 3) Leaderless (Dynamo-Style Quorum): Lider yoktur; istemci doğrudan N düğüme yazar ve okur. Quorum Kuralı: $R + W > N$. Eğer 3 replikalı bir kümede $W=2$ ve $R=2$ ise, okunan 2 düğümden en az biri en son yazmayı görmüş olmak zorundadır (Version Vector ile en günceli seçilir).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Single-Leader (Active-Passive): Writes direct strictly to the leader, streaming mutations via Write-Ahead Logs (WAL) to read-replicas. Horizontally scales read bandwidth, but asynchronous replication introduces replication lag and stale reads (mitigated via read-your-own-writes session sticky routing to leader). 2) Multi-Leader (Active-Active): Multiple datacenters accept local writes, requiring conflict resolution strategies like Last-Write-Wins (clock-skew sensitive) or CRDTs. 3) Leaderless Quorum (Amazon Dynamo, Cassandra): Clients coordinate writes across N nodes. The Quorum Equation: $R + W > N$. If N=3, W=2, and R=2, the read set and write set must overlap by at least one node, guaranteeing strong consistency via read-repair or hinted handoff.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Quorum Okuma ve 'En Güncel Versiyonu' Seçme Motoru
import java.util.List;

public class QuorumReadCoordinator {
    public record VersionedValue(String value, long versionTimestamp) {}

    public VersionedValue resolveQuorumRead(List<VersionedValue> responses) {
        // En yüksek zaman damgasına sahip veriyi seç (Last-Write-Wins)
        return responses.stream()
                .max((a, b) -> Long.compare(a.versionTimestamp(), b.versionTimestamp()))
                .orElseThrow();
    }
}

void main() {
    var coordinator = new QuorumReadCoordinator();
    // 3 Düğümden okunan 2 cevap (R=2):
    var node1 = new QuorumReadCoordinator.VersionedValue("User: Ali Bozkurt (Eski)", 1000L);
    var node2 = new QuorumReadCoordinator.VersionedValue("User: Ali Turgut Bozkurt (Yeni)", 1005L);

    var resolved = coordinator.resolveQuorumRead(List.of(node1, node2));
    IO.println("Quorum Çözümlenen Değer: " + resolved.value() + " [Versiyon: " +
resolved.versionTimestamp() + "]");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Asenkron replikasyonda 'Leader çöktüğünde veri kaybolmaz' demeyin! Asenkron replikasyonda Leader'a yazılan ancak henüz Follower'a iletilmeyen son işlemler Leader çökerse sonsuza kadar buharlaşır (Data Loss). Mülakatta yarı-senkron replikasyon veya Raft konsensüsü alternatiflerini tartışın.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

In asynchronous replication, if the leader crashes before WAL packets arrive at replicas, committed writes are permanently lost. Explicitly call out this failover data loss window and present semi-synchronous replication or Raft consensus as hardening options.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Single-Leader okuma ağırlıklı sistemlerde okumayı follower'lara yayarak ölçekler.
- R + W > N quorum formülü lidersiz sistemlerde güçlü tutarlılık sağlar.
- Replication Lag, kullanıcının kendi yaptığı güncellemeyi hemen görememesine yol açabilir.

🎯 Key Takeaways for the Interview

- Single-Leader topologies scale read-heavy workloads across fleets of replicas.
- The quorum condition R + W > N ensures read and write sets overlap on at least one fresh node.
- Replication lag causes stale read anomalies unless read-after-write consistency is enforced.

22

BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

Veritabanı Bölümleme (Sharding) ve Yatay Parçalama: Range, Hash ve Directory

Database Sharding & Horizontal Partitioning: Range, Hash & Directory

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram

```
[ 1. RANGE-BASED SHARDING (Aralık Tabanlı) ]
Shard 1: Kullanıcı A - H  |  Shard 2: Kullanıcı I - P  |  Shard 3: Kullanıcı Q - Z
(Sıralı sorgular hızlıdır; ancak 'J' harfi aşırı popülerse Shard 2 patlar - Hotspot!)

[ 2. HASH-BASED SHARDING (Karma Tabanlı) ]
MurmurHash3(User_ID) % 4
Key: "usr_101" ⟶ Hash: 3948192 ⟶ % 4 = Shard 0
Key: "usr_102" ⟶ Hash: 8192841 ⟶ % 4 = Shard 1
(Veriyi homojen dağıtır; ancak aralık sorguları (Range Query) imkansızlaşır!)
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Devasa bir kütüphanedeki 10 milyon kitabı tek bir odaya sığdıramadığınızda ne yaparsınız? Kütüphaneyi 3 odaya bölersiniz (Sharding): 1) Range Sharding (Harfe Göre): 1. odaya A-H arası yazarlar, 2. odaya I-P arası konur. Düzenlidir ama o sene 'Harry Potter' patlarsa H odasının kapısında izdiham çıkar (Hotspot), diğer odalar bomboş kalır! 2) Hash Sharding (Rastgele Matematiksel Dağıtım): Kitabın adını matematiksel bir mikserden (Hash fonksiyonu) geçirir ve odalara eşit dağıtırsınız. Odaların yükü dengelenir ama 'Bana 1990'lardaki tüm kitapları getir' derseniz 3 odayı da tek tek aramak zorunda kalırsınız!

🇬🇧 ENGLISH (ELI5)

When 10 million books overflow a library, you split them across 3 separate wings (Sharding): 1) Range Sharding: Wing 1 takes authors A-H, Wing 2 takes I-P, Wing 3 takes Q-Z. It keeps ranges alphabetical, but if J.K. Rowling blows up, Wing 2 is crushed under mob traffic (Hotspotting) while Wing 3 is dead empty! 2) Hash Sharding: You run the book ISBN through a mathematical blender (Hash function) and scatter copies evenly across all 3 wings. Traffic distributes uniformly, but executing a range query ('Find all books written between 1990 and 1995') forces you to search every single room!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Sharding (Yatay Bölümleme), tek bir veritabanının kaldıramayacağı büyüklükteki verileri ve QPS'i bağımsız veritabanı düğümlerine (Shard) bölmektir. Yöntemler: 1) Range-Based Sharding: ID veya tarihe göre aralıklarla bölme (ör. 1-100K Shard 1, 100K-200K Shard 2). Aralık sorguları (Range Scan) verimlidir; ancak sıralı ID'lerde en son shard sürekli sıcak kalır (Hotspot). 2) Hash-Based (Key-Based) Sharding: `Hash(ShardKey) % ShardSayısı`. Veriyi homojen dağıtır, hotspot'ları engeller; ancak shard sayısı değiştiğinde yeniden veri taşıma (Resharding) felaketi yaşanır (Bkz: Consistent Hashing). 3) Directory-Based: Merkezi bir arama tablosunda hangi anahtarın hangi shard'da olduğu tutulur; esnektir ama tablo tek hata noktası (SPOF) olabilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Sharding partitions massive datasets across autonomous database instances. Strategies: 1) Range-Based Sharding: Allocates discrete contiguous intervals (e.g. A-F, G-M, or date brackets). Excellent for range scans, but susceptible to non-uniform data skew and write hotspots on append-only primary keys. 2) Hash-Based Sharding: Computes `hash(shard_key) % num_shards`. Guarantees uniform distribution and squashes hotspots, but ruins range queries (requires scatter-gather across all shards) and triggers massive rebalancing overhead when expanding nodes. 3) Directory-Based Sharding: Uses a centralized lookup service mapping entity IDs to target shards; highly adaptable but introduces an operational lookup hop and potential bottleneck.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Hash Tabanlı Sharding Yönlendirici (Shard Router)
public class ShardRouter {
    private final int numberOfShards;

    public ShardRouter(int numberOfShards) {
        this.numberOfShards = numberOfShards;
    }

    public int resolveShardId(String shardKey) {
        // MurmurHash benzeri dağıtık pozitif hash simülasyonu
        int hash = shardKey.hashCode() & 0x7fffffff;
        return hash % numberOfShards;
    }
}

void main() {
    var router = new ShardRouter(4); // 4 Veritabanı Parçası (Shard 0, 1, 2, 3)
    IO.println("user_1001 -> Shard #" + router.resolveShardId("user_1001"));
    IO.println("user_1002 -> Shard #" + router.resolveShardId("user_1002"));
    IO.println("user_1003 -> Shard #" + router.resolveShardId("user_1003"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Sharding'in 3 büyük bedeli vardır: 1) Cross-shard JOIN imkansızlaşır (uygulama katmanında yapılmalıdır). 2) Dağıtık işlemler (Distributed Transactions / 2PC) korkunç yavaştır. 3) Shard Rebalancing (yeni makine ekleyip veriyi taşımak) operasyonel bir kabustur. Bu bedelleri mülakatta açıkça sayın!

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Articulate the 3 heavy penalties of sharding: 1) Cross-shard joins become computationally impossible. 2) Cross-shard transactions require agonizingly slow Two-Phase Commits (2PC). 3) Resharding data across new nodes is an operational minefield.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Sharding, tek bir makinenin disk ve IO limitleri aşıldığında zorunlu son çaredir.

• Hash-based sharding veriyi eşit dağıtır ancak aralık sorgularını (range query) zorlaştırır.

• Shard Key seçimi (kullanıcı ID vs şirket ID) mimarinin başarısını belirleyen en kritik karardır.

🎯 Key Takeaways for the Interview

• Sharding is the ultimate architectural frontier when single-node IO limits are exhausted.

• Hash sharding prevents write skew at the cost of requiring scatter-gather range queries.

• Selecting the Shard Key determines query locality and system longevity.

23

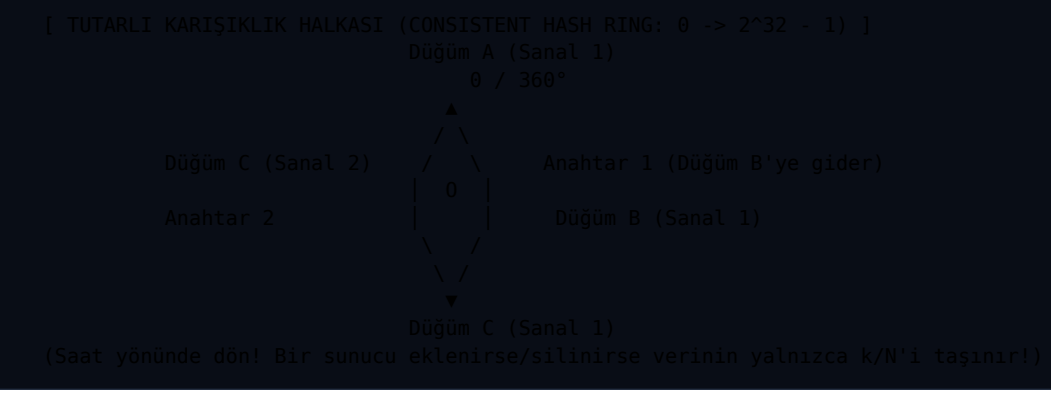
BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

Tutarlı Karışıklık (Consistent Hashing) & Sanal Düğümler (Virtual Nodes)

Consistent Hashing & Virtual Nodes: The Elastic Scaling Ring

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Geleneksel modülo aritmetiğinde (`Hash % N`), 4 sunucunuz varken 5. sunucuyu eklerseniz sınıftaki herkesin sırası değişir; tüm önbellek geçersiz kalır ve veritabanı çöker! Consistent Hashing ise yuvarlak bir yemek masası gibidir (0'dan 360 dereceye bir çember). Sunucuları masanın etrafına rastgele oturtursunuz. Yeni gelen bir yemeği masanın üzerine koyup saat yönünde döndürürsünüz; yemeği ilk gören sunucu onu afiyetle yer! Masaya yeni bir sandalye koysanız bile sadece o sandalyenin hemen yanındaki birkaç yemek yer değiştirir; geri kalan herkes yemeğini yemeye devam eder!

🇬🇧 ENGLISH (ELI5)

Under naive modular hashing (`hash % N`), expanding from 4 to 5 servers scrambles the destination of 80% of all keys, triggering a total cache wipeout! Consistent Hashing is an infinite circular conference table (0 to 2^32 - 1). You position servers at discrete degrees around the circle. To store a key, hash it to a point on the circle and walk clockwise: the first server node you bump into claims the key! If a new server pulls up a chair, only its immediate neighbor hands over a sliver of keys; the remaining 80% of data stays completely untouched!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Consistent Hashing (Karger ve arkadaşları, MIT 1997), dağıtık önbellek (Memcached, Akamai CDN) ve NoSQL (Amazon DynamoDB, Apache Cassandra) mimarilerinin kalbidir. Avantajı: Kümedeki düğüm sayısı N değiştiğinde, taşınması gereken anahtar sayısı geleneksel mimarideki gibi %100 değil, sadece ortalama k/N kadardır (k = toplam anahtar). Sanal Düğümler (Virtual Nodes / Vnodes): Fiziksel bir sunucunun halka üzerinde tek bir noktada durması 'Hotspot' ve dengesiz yük dağılımına yol açar. Çözüm: Her fiziksel sunucuya halka üzerinde 100-256 adet sanal etiket atanır (`NodeA-1`, `NodeA-2`...). Bu sayede yük çember boyunca mükemmel bir homojenlikle dağılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Consistent Hashing maps both servers and data keys onto a continuous hash ring (typically 0 to 2^32 - 1). Keys route clockwise to the first encountered server token. When node scaling occurs, only k/N keys migrate (k = total keys, N = server count), avoiding mass cache evictions. The Virtual Nodes (Vnodes) Innovation: A physical server represented by a single token risks severe load skew due to random clustering. By projecting each physical node into 100-256 virtual points across the ring perimeter, keys distribute uniformly across hardware proportional to host capacity.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Sanal Düğümlü (Vnodes) Consistent Hash Halkası Implementasyonu
import java.util.*;

public class ConsistentHashRing<T> {
    private final int numberOfReplicas; // Sanal düğüm sayısı
    private final SortedMap<Integer, T> circle = new TreeMap<>();

    public ConsistentHashRing(int numberOfReplicas, Collection<T> nodes) {
        this.numberOfReplicas = numberOfReplicas;
        for (T node : nodes) addNode(node);
    }

    public void addNode(T node) {
        for (int i = 0; i < numberOfReplicas; i++) {
            circle.put(hash(node.toString() + "-vnode-" + i), node);
        }
    }

    public T getNode(String key) {
        if (circle.isEmpty()) return null;
        int hash = hash(key);
        if (!circle.containsKey(hash)) {
            SortedMap<Integer, T> tailMap = circle.tailMap(hash);
            hash = tailMap.isEmpty() ? circle.firstKey() : tailMap.firstKey(); // Saat yönünde dön
        }
        return circle.get(hash);
    }

    private int hash(String val) {
        return val.hashCode() & 0x7fffffff;
    }
}

void main() {
    var ring = new ConsistentHashRing<String>(3, List.of("Server-A", "Server-B", "Server-C"));
    IO.println("Key 'user_9912' Hangi Sunucuda? -> " + ring.getNode("user_9912"));
    IO.println("Key 'order_441' Hangi Sunucuda?  -> " + ring.getNode("order_441"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta sanal düğümleri (Virtual Nodes) anlatmayı unutursanız mülakatçı 'Düğümler halkaya rastgele düşerse biri aşırı yüklenip çökmez mi?' diye soracaktır. Sanal düğümlerin varyansı düşürüp yükü homojenleştirdiğini doğrudan siz söyleyin.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Failing to proactively propose Virtual Nodes (Vnodes) invites the interviewer to challenge you on hash skew. Preemptively demonstrate that virtual tokens equalize ring dispersion and reflect asymmetric server hardware.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Consistent Hashing, sunucu ekleme/çıkarmada veri taşıma maliyetini k/N seviyesine indirir.
- Sanal Düğümler (Virtual Nodes) çember üzerindeki yükü mükemmel dengeler ve hotspot'u önler.
- DynamoDB, Cassandra ve Discord gibi devasa dağıtık sistemlerin temel omurgasıdır.

🎯 Key Takeaways for the Interview

- Consistent Hashing bounds key migration to k/N upon node churn, preserving cache stability.
- Virtual Nodes neutralize non-uniform distribution skew across heterogeneous machines.
- Serves as the structural backbone of Amazon DynamoDB, Apache Cassandra, and Discord.

24

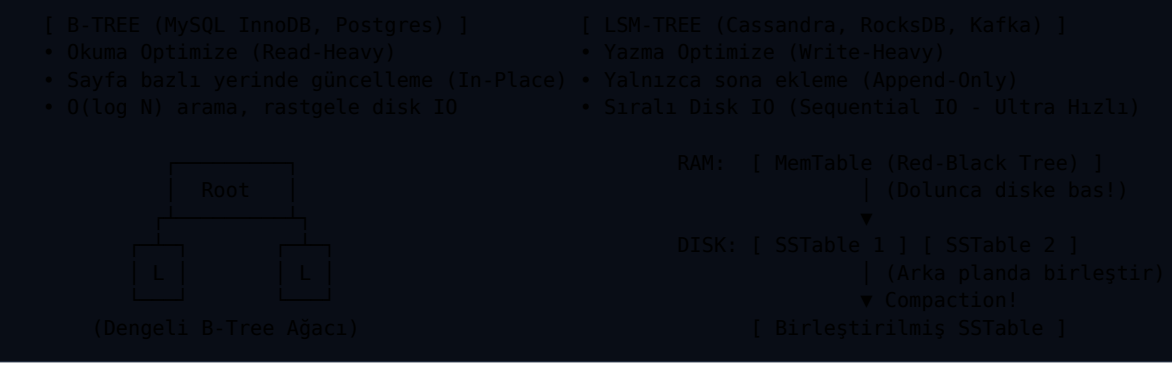
BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

Veritabanı İndeksleme Sırları: B-Tree vs LSM-Tree (Log-Structured Merge-Tree)

Database Indexing Under the Hood: B-Tree vs LSM-Tree

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

B-Tree, kalın ve çok düzenli bir kütüphane indeks fihristi gibidir: Yeni bir kitap geldiğinde fihristin ilgili sayfasına gidip araya sıkıştırarak yazarsınız (Yerinde Güncelleme - In-Place Update). Okurken harikadır, aradığınızı saniyesinde bulursunuz; ama çok fazla yeni kitap gelirse sayfaları silip yeniden düzenlemekten kütüphanecinin eli yorulur. LSM-Tree ise seri bir günlük yazarı gibidir: Asla eski sayfaları silmez! Ne olursa olsun günlüğün en sonuna yazar (Append-Only). Yazmak ışık hızındadır! Akşam olunca da arkada oturup günlükteki tekrarları temize çeker (Compaction)!

🇬🇧 ENGLISH (ELI5)

A B-Tree is a meticulously maintained card catalog in a library: When a new book arrives, the librarian navigates to the exact drawer and slots the card into alphabetical position (In-Place mutation). Reading is instantaneous ($O(log N)$), but heavy writes force costly random disk seeks. An LSM-Tree (Log-Structured Merge-Tree) is a court stenographer typing at 200 words/minute: They NEVER erase or rewrite past pages; they append every new event strictly to the end of the roll (Sequential Disk IO). Later, background janitors compress and deduplicate old logs into sorted tables (Compaction)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) B-Tree / B+Tree: İlişkisel veritabanlarının (MySQL InnoDB, PostgreSQL) kalbidir. Veri 16 KB'lık sayfalarda (Pages) dengeli bir ağaç yapısında tutulur. Sayfalar doğrudan yerinde (In-Place) güncellenir. Rastgele disk IO gerektirdiği için yazma maliyeti yüksektir; ancak okumalar tek bir sayfa aramasıyla tamamlanır ($O(log N)$). 2) LSM-Tree: Modern NoSQL ve depolama motorlarının (RocksDB, Cassandra, LevelDB) kalbidir. Yazmalar önce RAM'deki sıralı bir ağaca (`MemTable`) ve kurtarma günlüğüne (`WAL`) yazılır (sıfır rastgele disk IO!). MemTable dolunca diske sıralı ve değiştirilemez (Immutable) `SSTable` (Sorted String Table) olarak dökülür. Okuma yaparken: MemTable -> Bloom Filter -> SSTable dosyaları taranır. Arka planda `Compaction` işlemi eski sürümleri temizler.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) B-Tree / B+Tree: The classic relational storage engine (Postgres, InnoDB). Organizes fixed-size pages (typically 16KB) in a balanced tree. Updates perform random in-place page writes. Read paths are lightning fast ($O(log N)$ disk seeks), but high-velocity random writes fragment storage and cause heavy write amplification. 2) LSM-Tree (Log-Structured Merge-Tree): Powers modern high-throughput NoSQL engines (Cassandra, RocksDB, ClickHouse). Writes append sequentially to an in-memory sorted tree (`MemTable`) and a durable WAL, converting random writes into maximum-speed sequential IO. Saturated MemTables flush immutably to disk as `SSTables`. Background merge-compaction processes consolidate fragmented segments and evict deleted tombstones.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: LSM-Tree MemTable & SSTable Döküm Mantığı Simülasyonu
import java.util.*;

public class LsmTreeSim {
    // RAM'deki Sıralı Bellek Tablosu (MemTable)
    private final NavigableMap<String, String> memTable = new TreeMap<>();
    private final List<Map<String, String>> ssTablesOnDisk = new ArrayList<>();
    private static final int MEMTABLE_THRESHOLD = 3;

    public synchronized void put(String key, String value) {
        memTable.put(key, value); // RAM'e sıralı yaz (O(log N))
        if (memTable.size() >= MEMTABLE_THRESHOLD) {
            flushToSsTable();
        }
    }

    private void flushToSsTable() {
        // Diske sıralı, değiştirilemez SSTable olarak dök:
        ssTablesOnDisk.add(new HashMap<>(memTable));
        memTable.clear();
        IO.println(">>> MemTable Doldu! Diske SSTable döküldü (Sequential Disk IO). Toplam SSTable: "
+ ssTablesOnDisk.size());
    }

    public synchronized String get(String key) {
        // 1. Önce RAM'deki MemTable'a bak:
        if (memTable.containsKey(key)) return memTable.get(key);
        // 2. Bulunamazsa en yeni SSTable'dan eskiye doğru diski tara:
        for (int i = ssTablesOnDisk.size() - 1; i >= 0; i--) {
            if (ssTablesOnDisk.get(i).containsKey(key)) return ssTablesOnDisk.get(i).get(key);
        }
        return null;
    }
}

void main() {
    var lsm = new LsmTreeSim();
    lsm.put("user_1", "Ali");
    lsm.put("user_2", "Bob");
    lsm.put("user_3", "Can"); // Eşik doldu -> SSTable diske basıldı!
    lsm.put("user_4", "Deniz");

    IO.println("Sorgu 'user_1': " + lsm.get("user_1"));
    IO.println("Sorgu 'user_4': " + lsm.get("user_4"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

LSM-Tree'lerin zayıf karnı 'Read Amplification' (Okuma Büyümesi) ve 'Compaction' sırasında yaşanan disk IO darboğazlarıdır. Mülakatta bu dezavantajı gidermek için SSTable dosyalarının önüne mutlaka Bloom Filter konulduğunu belirtin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

LSM-Trees suffer from read amplification (checking multiple SSTable files on misses) and compaction IO storms. Explicitly state that pairing Bloom Filters with SSTables allows skipping unnecessary disk reads in 99% of queries.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• B-Tree okuma ağırlıklı sistemler (RDBMS) için, LSM-Tree ise yazma canavarları (NoSQL) içindir.

• LSM-Tree sıralı disk yazması (Sequential IO) kullanarak mekanik ve SSD diskleri son hızda sürer.

• Bloom Filter ve Compaction, LSM-Tree mimarisinin okuma performansını koruyan iki emniyet sibobudur.

🎯 Key Takeaways for the Interview

• B-Trees power read-heavy RDBMS; LSM-Trees dominate extreme write-heavy workloads.

• LSM-Trees convert random writes into blazing sequential append IO.

• Bloom Filters and background Compaction keep LSM-Tree read latencies tightly bounded.